

Hotel Reservation MLOps Project

Comprehensive Step-Wise Blueprint

Step 0: Executive Overview

Anticipating hotel-booking cancellations via a production-grade machine-learning pipeline. The project operationalises:

- Data & code versioning --- DVC & Git.
 - Experiment lineage --- MLflow.
 - Containerised inference --- Docker.
 - Automated CI/CD --- Jenkins --- Google Cloud Run.
-

Step 1: Business Objective & Problem Definition

Late-stage cancellations distort occupancy forecasts and revenue targets. This binary classifier predicts $is_canceled \in \{0,1\}$, enabling tactical levers dynamic pricing, controlled overbooking, and retention outreach.

Step 2: Data Foundation

2.1 Source Acquisition

- Dataset: Kaggle *Hotel Booking Cancellation* (~40 k rows, 31 features).
- Licence: CC BY-NC 4.0 --- verify non-commercial usage.

2.2 Data Governance & Version Control

- Raw CSV stored in data/raw/; tracked with DVC, remote on Cloud Storage.
- Processed frames in data/processed/; models in models/.

2.3 Exploratory Data Profiling (Jupyter)

- Notebook notebooks/eda.ipynb performs missingness heat-maps, distribution plots, and correlation matrices.
-

Step 3: Repository Topology & File Responsibilities

Path	Responsibility
src/	Python source modules (cleaning, feature engineering, model interfaces).
src/components/data_processing.py	Data cleansing, encoding, scaling, and split logic.
pipeline/	Orchestrated scripts; primary driver training_pipeline.py implements the full training DAG.
pipeline/params.yaml	Hyper-parameters, feature lists, and environment variables (read by Jenkins).
custom_jenkins/	Shared Groovy libraries for CI agents, Slack notifications, and GCR auth.
notebooks/	EDA, model prototyping, and interactive validation artefacts.
tests/	PyTest unit & integration tests ensuring pipeline integrity.
Dockerfile	Multi-stage Alpine image for the Flask micro-service.
Jenkinsfile	Declarative pipeline: Build → Test → Package → Deploy.
application.py	Flask entry-point exposing /predict REST/HTML endpoint.
templates/index.html	Simple HTML form for user predictions (User-App Building).
static/	CSS & JS assets for front-end.
requirements.txt	Python dependency manifest with pinned versions.
data/	DVC-tracked datasets (raw, processed).
reports/	Auto-generated Markdown & HTML diagnostics (e.g.
mlruns/	Local MLflow artifact store for experiments.

Step 4 — Environment & Infrastructure

4.1 Dependency Isolation

```
python -m venv venv && source venv/bin/activate
```

```
pip install -r requirements.txt # scikit-learn ≥ 1.7
```

4.2 Relational & Object Stores

- PostgreSQL for curated features & predictions (credentials via .env and Jenkins Secrets).
 - Google Cloud Storage bucket as DVC remote for large artefacts.
-

Step 5: Feature-Engineering Pipeline

1. Imputation --- median (numeric) / mode (categorical).
 2. Encoding --- one-hot for low-cardinality; target or frequency encoding for high-cardinality.
 3. Transformation --- log-scale avg_price_per_room, lead_time.
 4. Split --- stratified 80 / 20 (class-balanced).
-

Step 6: Model Development, Validation & Testing

6.1 Algorithm

- RandomForestClassifier for baseline; pluggable interface for XGBoost/LightGBM.

6.2 Hyper-parameter Optimisation

- Grid-search via GridSearchCV across n_estimators, max_depth, max_features.

6.3 Metric Suite

- Accuracy, Precision, Recall, F1, ROC-AUC.

6.4 Model Validation & Unit Testing

- tests/test_model.py verifies feature schema & prediction API.
-

Step 7: Experiment Management & Model Registry

- MLflow Tracking captures params, metrics, artefacts, and Git SHA.
- Best run auto-promoted to “Production” in MLflow Registry.

Step 8: Containerisation & User-App Build

1. Model Serialisation --- pickled via joblib to models/model.pkl.
 2. Flask Micro-service --- application.py + Jinja templates for user webform.
 3. Gunicorn Entrypoint --- gunicorn --bind 0.0.0.0:8080 application:app.
 4. Docker Build --- multi-stage, < 200 MB final image.
 5. Front-End Assets --- templates/ & static/ cover “USER APP BUILDING” node.
-

Step 9: Continuous Integration & Delivery (CI/CD)

9.1 Build

- Python agent → install deps → run PyTest → dvc pull.

9.2 Package

- Docker image tagged gcr.io/\$PROJECT/hotel-ml:\$BUILD_ID.

9.3 Deploy

- gcloud run deploy hotel-ml with 0-downtime rollout.

9.4 Artifact Retention

- Logs & images archived to Google Artifact Registry.
-

Step 10: Monitoring, Observability & Maintenance

- Drift Detection --- KS-stat > 0.20 triggers Slack alert.
 - Logging --- Cloud Run → Cloud Logging; metrics → Cloud Monitoring.
 - Canary Rollout --- 10 % traffic for 30 min; auto-rollback on error spike.
-

Step 11: Local Reproducibility Checklist

1 Clone repo

```
git clone https://github.com/Karthikmh2510/Hotel\_reservation\_MLOps\_project.git  
cd Hotel_reservation_MLOps_project
```

2 Setup env

```
cp .env.example .env # add DB creds  
python -m venv venv && source venv/bin/activate  
pip install -r requirements.txt  
dvc pull # datasets & model blobs
```

3 Run pipeline

```
python pipeline/training_pipeline.py
```

4 Serve locally

```
docker build -t hotel-ml:local .  
docker run -p 8080:8080 hotel-ml:local
```

Step 12: Future Road-Map

1. Upgrade REST layer to FastAPI for async throughput.
2. Add Feast feature store for on-line/off-line parity.
3. Elevate test coverage > 90 % (badge in README).
4. Integrate GitHub Actions for PR-level lint, security scans.